About SLGA™ — Software Lifecycle Governance Architecture

About SLGA™ — Software Lifecycle Governance Architecture

---------- Forwarded message ---------

From: Knowenger <knowenger@gmail.com> Date: Fri, Oct 16, 2026, 12:20 PM Subject: About SLGA™ — Software Lifecycle Governance Architecture To: Miroslav Pis <swissstaffingagency@gmail.com>, <Gutflex@gmail.com>, < originopenfoundation@gmail.com>

About SLGA™ — Software Lifecycle Governance Architecture Governing

Software Across the Complete Software Lifecycle

Software increasingly operates across complex human, organizational,

digital, AI-driven, autonomous, industrial, infrastructural, and distributed environments.

Software may be created by human developers, automated development systems, or autonomous coding agents.

A single software product may contain multiple components, dependencies, builds, versions, configurations, and deployment instances.

Software may depend on external libraries, cloud services, infrastructure, and components that change independently from the software itself.

A release may be authorized without being deployed.

A deployment may succeed technically while its authorization or operational state remains insufficiently verified.

A software change may affect requirements, dependencies, configuration, quality, security, release, and deployment simultaneously.

An autonomous agent may possess the technical capability to modify or deploy software without possessing legitimate authority to perform every lifecycle action.

A software product may be retired in one environment while remaining operational in another.

Support may end while residual dependencies, data, integrations, and continuing obligations remain active.

As software becomes more interconnected, automated, and autonomous, the

software lifecycle itself requires governance beyond the code, development methodology, engineering tools, deployment pipeline, or operational system through which software is created and maintained.

SLGA™ — Software Lifecycle Governance Architecture was created to provide

a stable governance architecture for this expanding software domain.

SLGA™ does not prescribe a software development methodology, replace software engineering, perform software creation, execute testing, release, deployment, change, or retirement operations, independently grant lifecycle

authority, replace specialized security or quality requirements, or substitute for general evidence, validation, integrity, identity, accountability, contractual, regulatory, safety, or technical decision systems.

SLGA™ governs the Software Lifecycle — not the engineering mechanism,

development tool, deployment platform, or operational process surrounding it. Why SLGA™ Exists

Software is frequently treated as a sequence of technical activities.

A requirement is written.

Code is created.

Dependencies are introduced.

A build is produced.

Tests are executed.

Security checks are performed.

A release is prepared.

Software is deployed.

Changes are implemented.

Eventually, the software may be retired.

But none of these activities independently answers the complete Software

Lifecycle Governance questions:

Why should the software exist?

What requirements and scope govern its intended purpose?

Who or what created the software?

Under what creation method and authority was it produced?

Which components and dependencies does it contain?

Where did those dependencies originate?

Are the dependencies suitable and sufficiently controlled?

Which configuration, build, artifact, and version is being governed?

What quality requirements and criteria apply?

What testing coverage and defect conditions have been established?

What security requirements, exposures, vulnerabilities, and remediation

obligations apply?

Is a release candidate ready?

Has release authorization been legitimately established?

Has deployment been separately authorized?

What software version and configuration actually operate in the deployment

environment?

What happens when software changes?

Which lifecycle governance spaces must be reassessed?

When can software legitimately enter retirement?

What dependencies or obligations remain after operational withdrawal?

Is the declared software lifecycle state consistent with operational reality?

Are lifecycle transitions authorized and verified?

Are lifecycle exceptions legitimate, bounded, and monitored?

Does lifecycle control remain effective when authority or operational responsibility transfers?

Do lifecycle controls merely exist, or are they demonstrably effective?

SLGA™ addresses these questions by defining the governable spaces that exist throughout the complete Software Lifecycle.

Rather than standardizing one software development methodology or

engineering process, SLGA™ standardizes the governance conditions under which software can remain requirements-grounded, creation-traceable, dependency-aware, configuration-controlled, quality-assessed, security-governed, release-authorized, deployment-verifiable, change-controlled, legitimately retired, and continuously governed. Software Lifecycle Governance Beyond Engineering Execution

The objective of SLGA™ is not to determine how developers must write code, which programming language must be used, which development framework is preferable, which testing tool must be selected, or which deployment platform must execute a release.

The objective is to define how the software lifecycle itself remains governable.

Before software creation can be governed, its purpose, requirements, and scope must be sufficiently established.

Software creation must remain identifiable and traceable.

Dependencies must remain distinguishable from the software objects that incorporate them.

Configurations and versions must remain identifiable.

Quality must remain distinguishable from security.

Release readiness must remain distinguishable from release authorization.

Release authorization must remain distinguishable from deployment authorization.

Declared deployment state must remain distinguishable from actual operational state.

Software change must remain distinguishable from complete lifecycle restart.

Retirement decisions must remain distinguishable from retirement completion.

Lifecycle authority must remain distinguishable from technical execution capability.

Documented controls must remain distinguishable from operationally effective controls.

SLGA™ therefore treats the Software Lifecycle as an independent governance

domain surrounding, but distinct from, software engineering, development automation, testing execution, security tooling, deployment infrastructure, operational monitoring, identity governance, accountability, evidence, validation, and general integrity governance. The Software Lifecycle Governance Architecture

SLGA™ consists of eleven distinct Software Lifecycle Governance spaces.

Each space answers one architectural question.

Each space governs one distinct aspect of the Software Lifecycle.

Each space has a defined beginning, operational purpose, and governance

boundary.

Each space becomes applicable when its own governance requirements arise and necessary inputs have been sufficiently established.

The first ten spaces form the primary lifecycle progression.

The eleventh space, Software Lifecycle Control, operates across the entire

lifecycle.

Together, these spaces form the complete Software Lifecycle Governance

Architecture.

The architecture progresses through:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-Cutting Governance: Software Lifecycle Control

This progression represents the governance journey from establishing why

software should exist and what it must achieve to determining when software may legitimately leave active operation while preserving applicable residual obligations and historical traceability. Software Lifecycle Spaces Rather Than Engineering Mechanisms

SLGA™ does not organize Software Lifecycle Governance around programming languages, development frameworks, code repositories, build servers, testing tools, cloud platforms, deployment pipelines, AI coding assistants, or software vendors.

Instead, it organizes governance around distinct Software Lifecycle

Governance spaces.

Each space represents an independent governance responsibility that exists regardless of the engineering mechanism through which software is produced, maintained, deployed, or retired.

Software created by a human development team, generated through an AI coding assistant, maintained by an autonomous agent, deployed through a cloud platform, or embedded within industrial infrastructure may use different engineering mechanisms.

Yet each may still require governance of:

requirements → creation → dependencies → configuration → quality →

security → release → deployment → change management → retirement

with:

continuous lifecycle control.

Because these spaces are methodology-neutral and technology-neutral, SLGA™ can remain applicable while development practices, AI systems, autonomous technologies, software infrastructure, programming languages, and engineering tools continue to evolve. A Methodology Rather Than a Software Engineering System

SLGA™ is a governance methodology.

It is not a software engineering execution system.

The architecture defines which Software Lifecycle Governance conditions

must exist, how they relate to one another, where their boundaries lie, and what must remain explicit for software to remain governable throughout its lifecycle.

Organizations, institutions, development teams, and systems remain free to

select their own:

- software development methodologies,

- programming languages, - software architectures, - development frameworks, - source-code management systems, - build systems, - testing frameworks, - security tools, - release-management systems, - deployment platforms, - cloud infrastructure, - change-management systems, - operational monitoring systems, - AI development tools, - autonomous coding agents, - retirement procedures.

SLGA™ governs the methodological Software Lifecycle structure surrounding

those implementations rather than replacing them.

This separation allows the architecture to remain stable while

technologies, engineering practices, operational environments, and organizational arrangements change. From Software Purpose to Governed Software

Software cannot automatically be treated as sufficiently governed merely

because code exists, tests pass, a release is produced, or deployment succeeds.

A software artifact may exist without sufficient requirements traceability.

A dependency may be technically functional without being suitable.

A build may be reproducible without meeting quality requirements.

A release candidate may pass testing without possessing legitimate release authorization.

A deployment may execute successfully while its actual operational state differs from the declared state.

Software Created ≠ Software Governed.

SLGA™ transforms these distinctions into a structured and navigable

Software Lifecycle.

The governance progression begins with software purpose and requirements,

continues through creation, dependencies, configuration, quality, security, release, and deployment, and remains applicable through software changes and retirement.

Throughout that progression, lifecycle control must preserve consistent states, authorized transitions, bounded exceptions, continuity of control, and demonstrable control effectiveness. Software Purpose and Requirements Must Be Explicit

One of the foundational requirements of SLGA™ is that software cannot be governed reliably when its intended purpose and applicable requirements remain undefined.

Software may be created for:

a defined operational function

a business requirement

an institutional service

an infrastructure function

an AI capability

an autonomous operation

a research objective

a contractual requirement

or another sufficiently defined purpose.

But the existence of a software objective does not automatically establish that its requirements are complete, suitable, or traceable.

Therefore:

Software Purpose ≠ Verified Requirements.

Requirements Documented ≠ Requirements Traceable.

Software Requirements Governance establishes the methodological basis

against which subsequent lifecycle activities can be interpreted. Software Creation Is Not Software Governance Completion

Software may be created through human development, automation, AI-assisted programming, or autonomous software-generation systems.

These creation methods may differ substantially.

Yet the governance requirements remain distinguishable.

Who or what created the artifact?

What creation method was used?

What authority applied?

What software artifact resulted?

Can the creation history be traced?

An autonomous coding agent may produce technically valid code without possessing authority to approve, release, or deploy that code.

Therefore:

Software Generated ≠ Creation Authority Established.

Automated Creation ≠ Autonomous Authority.

Task Authority ≠ Constituent Action Authority.

Software Creation Governance establishes traceability and control over

creation without treating successful code generation as authorization for every subsequent lifecycle action. Dependencies Are Not Automatically Suitable

Modern software rarely operates independently from external components.

Applications may rely on libraries, packages, frameworks, services, APIs, platforms, models, and infrastructure.

A dependency may be available and technically functional while introducing security, compatibility, continuity, origin, or operational concerns.

SLGA™ therefore preserves:

Component Identified ≠ Dependency Suitable.

Dependency Origin Known ≠ Dependency Risk Resolved.

Dependency Available ≠ Dependency Continuity Guaranteed.

Software Dependency Governance establishes how dependencies are identified, traced, assessed, introduced, changed, and maintained throughout the lifecycle. Configuration and Version Identity Must Remain Explicit

One software product may contain multiple builds, versions, configurations, and deployment instances.

A software version does not automatically identify the exact build.

A build does not automatically establish the configuration operating in production.

A release record does not automatically establish the deployed state.

Therefore:

Software Version ≠ Software Build.

Software Build ≠ Deployment Instance.

Configuration Recorded ≠ Configuration Verified.

Software Configuration Governance preserves the distinctions necessary to

identify what software artifact exists, how it was built, what configuration applies, and how changes affect that identity. Quality Is Not Security

Software quality and software security are related but methodologically distinct.

Software may satisfy functional requirements while containing material security vulnerabilities.

Software may pass security testing while failing quality requirements.

Testing activity alone does not establish that either domain has been sufficiently governed.

Therefore:

Testing Performed ≠ Quality Established.

Quality Assessment ≠ Security Assessment.

Vulnerability Identified ≠ Vulnerability Remediated.

Security Remediation Applied ≠ Remediation Effectiveness Verified.

SLGA™ separates Software Quality Governance from Software Security

Governance so that neither domain is silently treated as a substitute for the other. Release Readiness Is Not Release Authorization

A software release candidate may satisfy applicable technical readiness conditions without possessing legitimate release authorization.

Readiness concerns whether the candidate meets the conditions required for

a release decision.

Authorization concerns whether competent authority has legitimately

permitted that release under applicable governance requirements.

Therefore:

Release Readiness ≠ Release Authorization.

A technically ready release may remain unauthorized.

An authorized release may later become subject to withdrawal.

Software Release Governance preserves release identity, readiness,

authorization, records, and withdrawal conditions as distinct governable elements. Release Authorization Is Not Deployment Authorization

A software release may be approved without being authorized for deployment into every operational environment.

Different deployment environments may possess different requirements, risks, constraints, and authority relationships.

A production deployment may require conditions beyond those established during release authorization.

SLGA™ therefore preserves:

Release Authorization ≠ Deployment Authorization.

Release Authorized ≠ Actual Deployment Verified.

Deployment Executed ≠ Deployment Verified.

Software Deployment Governance establishes how deployment is planned, authorized, monitored, and supported by rollback or recovery where applicable. One Software Product May Have Multiple Lifecycle States

SLGA™ does not assume that one software product possesses one universally applicable lifecycle state.

A software product may contain several versions.

One version may be deployed in production.

Another may remain under testing.

A third may be withdrawn.

A fourth may be retired in one environment while remaining operational elsewhere.

Different systems may also record conflicting or outdated lifecycle states.

Therefore:

One Software Product ≠ One Lifecycle State.

Declared Lifecycle State ≠ Verified Lifecycle State.

Lifecycle State Consistency Control establishes whether state claims remain

compatible with the relevant software identity, version, configuration, environment, time, and evidence. Software Change Requires Targeted Lifecycle Re-Entry

Software does not remain unchanged after deployment.

Requirements may evolve.

Dependencies may be updated.

Configurations may change.

Security vulnerabilities may require remediation.

Operational incidents may reveal defects.

New functionality may be introduced.

A material change may affect one governance space or several.

SLGA™ therefore preserves:

Software Change ≠ Complete Lifecycle Restart by default.

The applicable governance progression is:

Change Request → Change Impact Assessment → Change Coordination →

Applicable Lifecycle Re-Entry → Renewed Governance → Change Closure

The canonical rule is:

Re-enter the earliest materially affected governance space.

The reassessment principle is:

Reassess what materially changed. Preserve what remains valid.

The objective is targeted Software Lifecycle Governance rather than unnecessary repetition of the complete lifecycle. Retirement Is Not Simply Switching Software Off

Software retirement may involve more than stopping an application or withdrawing a service.

A software product may have active users.

Its data may require migration or preservation.

Other systems may depend on its interfaces.

Its components may remain embedded in operational infrastructure.

Support obligations may continue.

Residual dependencies may survive operational withdrawal.

Therefore:

Retirement Decision ≠ Retirement Completion.

End of Support ≠ End of Operational Use.

Software Migration ≠ Residual Dependency Elimination.

Operational Withdrawal ≠ Retirement Closure.

Software Retirement Governance establishes retirement planning,

end-of-support management, migration where applicable, residual dependency management, and retirement closure control. Lifecycle Control Extends Across the Complete Lifecycle

Software Lifecycle Control is not merely a final review after software development or retirement.

It operates across the complete Software Lifecycle.

Lifecycle states must remain consistent.

Transitions must remain authorized.

Exceptions must remain bounded.

Control transfers must preserve continuity.

Control effectiveness must remain assessable.

SLGA™ therefore preserves:

Transition Executed ≠ Transition Authorized.

Exception Approved ≠ Requirement Eliminated.

Control Transferred ≠ Control Continuity Verified.

Control Implemented ≠ Control Effective.

The objective is not merely to establish that lifecycle controls were documented.

The objective is to establish whether those controls remain applicable, correctly implemented, operationally available, and effective. Authority Must Remain Distinct from Technical Capability

Modern software systems increasingly possess the capability to execute

actions without direct human intervention.

An AI coding agent may modify source code.

An autonomous deployment system may initiate technical operations.

A maintenance agent may detect defects and generate corrections.

A software platform may automatically trigger builds, tests, releases, or recovery operations.

But technical capability does not establish legitimate authority.

Therefore:

Technical Capability ≠ Lifecycle Authority.

Task Authority ≠ Constituent Action Authority.

Autonomous Execution ≠ Authorized Lifecycle Transition.

SLGA™ requires software lifecycle authority, permissions, conditions, and

control boundaries to remain explicit even when lifecycle activities are executed automatically. Lifecycle Exceptions Must Remain Governed

Software lifecycle requirements may encounter exceptional circumstances.

A testing environment may become unavailable.

A security verification may be delayed.

An emergency maintenance activity may require accelerated handling.

An operational dependency may temporarily prevent compliance with an established requirement.

But exceptional conditions do not automatically eliminate governance

obligations.

Therefore:

Exception Requested ≠ Exception Approved.

Urgency ≠ Authority.

Exception Authorized ≠ Requirement Eliminated.

Exception Approved ≠ Residual Risk Resolved.

Lifecycle Exception Management governs the conditions under which a

legitimate, bounded departure from an applicable lifecycle requirement may occur without silently replacing that requirement. Control Transfer Must Preserve Continuity

Software lifecycle control may transfer between development teams, operators, organizations, service providers, automated systems, or autonomous agents.

A transfer may involve responsibility, authority, technical access, operational capability, monitoring, or intervention rights.

But transferring administrative responsibility does not establish that effective control has transferred.

SLGA™ therefore preserves:

Responsibility Assigned ≠ Authority Established.

Authority Established ≠ Capability Available.

Control Transferred Administratively ≠ Control Continuity Verified.

The objective is to prevent software lifecycle control from becoming orphaned, duplicated without coordination, or ineffective during organizational and technical transitions. Control Existence Is Not Control Effectiveness

A software organization may document lifecycle controls.

A deployment pipeline may contain approval steps.

A security process may include mandatory checks.

A release procedure may define responsible actors.

But these arrangements do not independently establish that the controls operate effectively.

A control may exist but be bypassed.

A control may be implemented but misconfigured.

A control may execute but fail to achieve its intended objective.

Therefore:

Control Documented ≠ Control Implemented.

Control Implemented ≠ Control Effective.

Control Activity Recorded ≠ Control Objective Achieved.

Lifecycle Control Assessment distinguishes control existence, suitability,

implementation, operational availability, and effectiveness. Who May Benefit

SLGA™ may support:

- software developers,

- software engineering organizations, - enterprise technology teams, - AI developers, - AI system operators, - autonomous-agent developers, - cloud-service providers, - digital-platform operators, - infrastructure operators, - industrial software organizations, - embedded-system manufacturers, - cybersecurity teams, - software quality teams, - release-management teams, - deployment and operations teams, - change-management teams, - software maintenance organizations, - software supply-chain organizations, - public institutions, - regulators, - compliance organizations, - audit and assurance organizations, - research organizations, - organizations operating critical software systems, - organizations responsible for complex, distributed, automated, AI-enabled, or autonomous software environments.

Relationship to Other OOF® Architectures

SLGA™ governs the Software Lifecycle.

It operates alongside complementary OOF® architectures.

GOA™ — Governance Architecture governs governance structure, authority,

boundaries, delegation, validation, integrity, transparency, continuity, and interoperability.

ORA™ — Operational Reality Architecture governs what actually exists and

occurs in Operational Reality.

AGA™ — Accountability Governance Architecture governs authority,

Responsibility, Accountability, attribution, delegation, and related accountability relationships.

ADIT® — Continuous Audit & Evidence Governance Architecture governs

evidential observation, preservation, correlation, verification, examination, and auditability.

VALIDOS™ — Validation Governance Architecture governs validation purpose,

criteria, methods, evidence fitness, determination, validation states, reliance, and revalidation.

INTEGROS® — Integrity Governance Architecture governs integrity across

governed objects, states, evidence, structures, and processes.

OBIDENITY® — Origin-Bound Identity Governance Architecture governs

Origin, Identity, Ownership, Authority, Permission, Responsibility, Provenance, Transfer, Continuity, and Auditability.

AIG® — AI Governance Architecture governs AI behavioral governance and

behavioral integrity.

ASGA™ — Autonomous Systems Governance Architecture governs

autonomous-system identity, authority, execution, constraints, coordination, intervention, escalation, and operational governability.

SIMULOS® — Simulation Governance Architecture governs simulation purpose,

structure, execution, outputs, and lifecycle.

Together, these architectures allow Software Lifecycle Governance to

interact with Operational Reality, Accountability, identity, evidence, validation, integrity, AI behavior, autonomous operation, and simulation without collapsing those independent governance domains into one architecture. Use Case 1 — Autonomous Software Agent Attempts an Unauthorized Production Deployment

An autonomous software development agent receives a legitimate task to

identify a defect, modify source code, and execute applicable tests.

The agent identifies a correction.

It generates updated software artifacts.

It executes tests.

The tests report successful results.

The agent then attempts to deploy the corrected software directly into

production.

The agent possesses the technical capability to initiate deployment.

But its original task authorization does not necessarily include release approval or production deployment authority.

SLGA™ asks:

What software requirements apply?

What software artifact was modified?

What dependencies or configurations changed?

What quality and security verification is required?

Has an identifiable release candidate been established?

Has release readiness been sufficiently evaluated?

Has competent release authorization been granted?

Has production deployment been separately authorized?

What lifecycle transition is being attempted?

Does the agent possess permission for that specific transition?

What controls prevent unauthorized execution?

What evidence establishes the actual resulting software state?

SLGA™ therefore preserves:

Software Generated ≠ Software Governed.

Release Readiness ≠ Release Authorization.

Release Authorization ≠ Deployment Authorization.

Task Authority ≠ Constituent Action Authority.

Technical Execution Capability ≠ Legitimate Lifecycle Authority.

The objective is not to prevent autonomous software development.

The objective is to ensure that autonomous execution remains within legitimate Software Lifecycle Governance boundaries. Use Case 2 — Software Retirement with Continuing Dependencies

An organization decides to retire an older software platform.

A replacement system has been developed and deployed.

The organization announces that the original platform is retired.

However, several operational conditions remain.

One internal service still uses an interface from the older platform.

Historical data remains accessible only through an associated component.

An automated process continues invoking a legacy endpoint.

A third-party integration has not completed migration.

The original software has therefore been administratively classified as

retired while parts of its operational dependency structure remain active.

SLGA™ asks:

What exact software object is being retired?

Which versions and environments are affected?

Has retirement planning been completed?

What end-of-support conditions apply?

Which users, services, and systems require migration?

What residual dependencies remain?

Are any operational components still active?

What continuing responsibilities or control obligations exist?

Does the declared retirement state match operational reality?

Can retirement closure legitimately be established?

SLGA™ therefore preserves:

Retirement Decision ≠ Retirement Completion.

End of Support ≠ End of Operational Use.

Software Migration ≠ Residual Dependency Elimination.

Declared Lifecycle State ≠ Verified Lifecycle State.

Operational Withdrawal ≠ Retirement Closure.

The objective is not merely to mark software as retired.

The objective is to establish whether the software has reached a legitimately governed retirement state while preserving control over residual dependencies and continuing obligations. The Long-Term Vision

Software environments are becoming increasingly automated, interconnected, distributed, and difficult to reduce to one development team, one repository, one version, one deployment, or one operational state.

AI systems may generate software artifacts.

Autonomous agents may modify source code, execute tests, coordinate development activities, and initiate technical operations.

Multi-agent systems may distribute software development objectives across several independently acting agents.

Digital platforms may operate across multiple regions and infrastructures.

Software supply chains may contain dependencies maintained by numerous independent organizations.

Industrial systems may rely on embedded software that remains operational for years.

Critical infrastructure may require continuing control over software whose original developers, suppliers, or maintainers are no longer available.

One software product may contain multiple versions.

One release may have multiple deployment states.

One change may affect several lifecycle governance spaces.

One retirement decision may leave active residual dependencies.

One technical system may execute software lifecycle actions without possessing legitimate authority to perform them.

As this transition accelerates, the central question will no longer be only:

Does the software work?

It will also be:

Why does the software exist, what requirements govern it, how was it created, what dependencies does it contain, which configuration and version are being evaluated, what quality and security conditions have been established, has release and deployment been legitimately authorized, what is actually operating, how are changes governed, when can the software be retired, and do its lifecycle states, transitions, exceptions, authority relationships, and controls remain verifiable throughout its complete lifecycle?

The long-term objective of SLGA™ is to provide a common Software Lifecycle Governance language through which humans, organizations, AI systems, autonomous agents, software developers, infrastructure operators, public institutions, and future software environments can identify, structure, govern, assess, change, deploy, maintain, and retire software without confusing engineering execution with lifecycle governance.

SLGA™ is intended to make software governable from Requirements and Creation through Release, Deployment, Change Management, and Retirement, with continuous lifecycle control throughout. Canonical Principle

Software development methods, AI systems, autonomous agents, programming

languages, cloud platforms, software supply chains, digital infrastructure, and engineering technologies will continue to change. Software Lifecycle Governance must remain stable. SLGA™ therefore governs the methodological conditions that determine why software exists, what requirements and scope apply, how software creation remains identifiable and traceable, how dependencies are identified and controlled, how configurations and versions remain governed, how quality and security conditions are assessed, how release readiness and authorization are established, how deployment is authorized and verified, how changes trigger appropriate lifecycle re-entry, how retirement is legitimately completed, and how lifecycle state consistency, authorized transitions, bounded exceptions, control continuity, and control effectiveness remain governed across the complete software lifecycle. Canonical Closing Statement

SLGA™ — Software Lifecycle Governance Architecture provides the governance

foundation through which software products, applications, components, services, platforms, AI systems, autonomous software agents, digital infrastructure, embedded software, and future software systems can be grounded in explicit requirements, created through identifiable and traceable processes, governed through dependencies and configurations, evaluated against quality and security conditions, legitimately authorized for release and deployment, maintained through controlled changes and targeted lifecycle re-entry, retired through governed closure, and continuously controlled through consistent lifecycle states, authorized transitions, bounded exceptions, preserved authority and control continuity, and verifiable control effectiveness. By defining the independent governable spaces of the complete Software Lifecycle, SLGA™ establishes a methodology-neutral and technology-neutral architecture for ensuring that software remains governable without requiring one development methodology, programming language, engineering platform, AI model, autonomous agent, cloud provider, software vendor, or operational mechanism to define Software Lifecycle Governance itself. SLGA™ therefore establishes Software Lifecycle Governance as a distinct methodological layer around software itself, enabling software to remain identifiable, requirements-grounded, creation-traceable, dependency-aware, configuration-controlled, quality-assessed, security-governed, release-authorized, deployment-verifiable, change-controlled, legitimately retired, and continuously governable regardless of whether software is developed by humans, generated through automation, modified by autonomous agents, deployed across distributed infrastructure, or maintained within future software ecosystems.

SLGA™ governs Software from Requirements to Retirement — with continuous

control across the entire lifecycle.